

SplitBench: Benchmarking Tool for Split Learning and Split Federated Learning

Samuel Trepac

Computing Sciences and Mathematics
Mount Royal University
Calgary, Canada
strep688@mtroyal.ca

Joseph Mills

Computing Sciences and Mathematics
Mount Royal University
Calgary, Canada
jmill004@mtroyal.ca

Yasaman Amannejad

Computing Sciences and Mathematics
Mount Royal University
Calgary, Canada
yamannejad@mtroyal.ca

Abstract—Systematic evaluation of split learning (SL) and split federated learning (SFL) is hindered by fragmented implementations that are often tied to a single algorithm, lack standardized data generation, or couple training logic to specific models and datasets. In this paper, we present SplitBench, an open-source framework for reusable benchmarking that consolidates sequential and parallel SL baselines, canonical SplitFed architectures (SFL-V1 and SFL-V2), and Custom Cut SFL (CC-SFL) extensions within a single configurable platform. The framework decouples architecture, model, and dataset selection from training logic, enabling comparison of algorithms under identical experimental conditions without re-implementation. It provides eight data partitioning strategies with distribution visualization, structured diagnostic logging, and containerized deployment. By emphasizing modularity, extensibility, and reproducibility, SplitBench supports reusable experimental workflows for comparative evaluation across diverse SL and SFL settings. The framework and experimental artifacts are publicly released to support reproducible benchmarking across the SL and SFL research landscape.

Index terms— Split Learning, Split Federated Learning, Reusable Benchmarking, Software Reusability, Distributed Machine Learning.

I. INTRODUCTION

Distributed machine learning enables collaborative model training across devices and organizations without centralizing raw data, a requirement in settings constrained by privacy regulations, institutional boundaries, and limited connectivity. Federated Learning (FL) supports this objective by coordinating parallel local training and periodically aggregating client updates at a server, but it typically requires each client to host and optimize the full model, which can be prohibitive for resource-constrained or heterogeneous devices. Split Learning (SL) reduces client-side computation by partitioning the model at a cut layer, allowing clients to compute only early layers and offload the remaining computation to a server. However, common multi-client SL deployments rely on relay-style sequential execution, leading to poor scalability as the number of clients increases.

Split Federated Learning (SFL) combines split execution with federated-style synchronization to reduce relay overhead while retaining the computational benefits of model partitioning. Existing SFL implementations, however, often assume homogeneous clients and enforce a single cut layer across all participants, limiting applicability in realistic edge

and healthcare deployments where client resources vary substantially. Recent algorithmic work has introduced custom-cut SFL variants that allow clients to adopt individualized cut layers, but tool support for systematic, reproducible evaluation across SL and SFL families remains limited. Available implementations are frequently tied to a single algorithm, lack standardized non-IID data generation and visualization, or couple training logic to specific models and datasets, making fair comparison, reusable experimentation, and reproducibility difficult.

This paper presents SplitBench, an open-source framework for split learning and split federated learning that consolidates multiple SL/SFL variants within a single configurable codebase. The full code repository is available online [1]. The framework implements sequential and parallel SL baselines, canonical SFL-V1 and SFL-V2, and heterogeneity-aware extensions that support client-specific cut layers under joint system and data heterogeneity. It decouples architecture, model, and dataset selection from training logic, supports controlled construction and visualization of non-IID partitions, and provides structured logging and containerized execution. Designed as reusable experimental infrastructure, SplitBench enables systematic and reproducible benchmarking across diverse SL and SFL settings. We demonstrate its capabilities through a representative set of experiments that illustrate controlled comparison across data heterogeneity, model choice, and cut-layer configurations.

The remainder of the paper is organized as follows. Section II reviews related work. Section III presents the design and implementation of SplitBench. Section IV demonstrates the framework through representative experiments, and Section V concludes the paper and outlines future work.

II. BACKGROUND AND RELATED WORK

This section reviews the main concepts and representative literature in split learning and split federated learning, with emphasis on architectural variants, heterogeneity, and existing benchmarking frameworks. Because SplitBench is designed as reusable benchmarking infrastructure, we also summarize prior tools that support comparative evaluation and reproducibility.

A. Split Learning and variants

Split Learning (SL), also known as SplitNN, partitions a neural network between a client and a server at a cut layer [2]. Clients execute forward propagation up to the cut layer and transmit intermediate activations (smashed data) to the server, which completes the forward pass, computes the loss and backpropagates gradients to the cut layer. The server returns cut-layer gradients so clients can complete local backpropagation. SL reduces client-side computation and memory requirements because only the prefix of the model is executed on clients [2].

Multi-client SL is commonly implemented using relay-style sequential execution, in which clients take turns interacting with a shared split server. While conceptually simple, relay execution increases wall-clock time as the number of participating clients grows [3], [4]. To address this bottleneck, parallel split learning formulations execute multiple clients concurrently and introduce systems concerns such as scheduling, effective batch size, and stability under heterogeneous resources [5], [6]. SL design also varies by label placement and execution topology, including U-shaped formulations studied in edge-assisted environments [7], [8].

B. Split Federated Learning and Extensions

Split Federated Learning (SFL) integrates split execution with federated-style synchronization to enable scalable multi-client training while retaining model partitioning benefits. The canonical SplitFed formulation distinguishes two widely used variants, SFL-V1 and SFL-V2 [4], [9]. In SFL-V1, both the client-side and server-side model segments are synchronized via federated aggregation at round boundaries, enabling parallel client execution while maintaining split computation. In SFL-V2, client-side segments are aggregated, whereas the server-side segment is updated sequentially using smashed data streams, avoiding explicit server-side aggregation [4]. These variants reflect different trade-offs in parallelism, server memory footprint, and synchronization cost. Privacy-enhanced SFL variants further incorporate robustness or privacy-preserving mechanisms [4], [10]. Our recent work has addressed client heterogeneity and cut-layer selection more explicitly [11], [12], increasing the need for reusable evaluation frameworks that can compare homogeneous and heterogeneous configurations under a shared setup.

C. Data heterogeneity and controlled non-IID evaluation

Statistical heterogeneity is a pervasive challenge in FL, SL, and SFL. Differences in label distributions, feature distributions, and sample counts across clients can bias local updates, slow convergence, and reduce final model quality [13]–[17]. Controlled partitioning is therefore essential for fair evaluation across architectures. Tools such as FedArtML further highlight the importance of controlled non-IID data generation by supporting reusable construction of federated datasets with multiple forms of heterogeneity, including label skew, feature skew, and quantity skew [18].

Dirichlet partitioning is widely used to generate tunable non-IID distributions via a concentration parameter α , where larger values yield more homogeneous partitions and smaller values produce stronger skew. Divergence metrics such as Jensen–Shannon divergence, Hellinger distance, and Earth Mover distance are commonly used to quantify non-IID severity. For benchmarking-oriented research, these controls improve comparability and reproducibility across algorithms.

D. Benchmarks, frameworks, and implementation landscape

The growing diversity of SL and SFL paradigms has motivated benchmarking efforts that standardize implementations and experimental protocols. Table I summarizes representative frameworks for SL/SFL experimentation. SLPerf proposes a unified benchmarking library for multiple SL paradigms and reports systematic comparisons under IID and non-IID regimes [20]. A later review and experimental evaluation further systematizes SL variants and reports engineering takeaways using SLPerf [26]. For SFL, the SplitFed reference implementation provides baselines for SL, FL, SFL-V1, and SFL-V2 and is widely used for comparative experimentation [4], [19].

SplitBud emphasizes modularity by implementing split learning abstractions on top of Flower [21], [22], [24]. More broadly, Flower and FedML provide orchestration substrates that can be extended to SL and SFL workflows [24], [25]. PyVertical addresses vertical SplitNN settings with private set intersection for entity alignment [23]. Production-oriented platforms such as FATE support collaborative learning in cross-silo settings, but are not optimized for head-to-head comparison across many SL and SFL variants [27].

Despite this progress, important gaps remain for reusable SL/SFL benchmarking. Many implementations focus on a narrow subset of algorithms, assume homogeneous clients, or provide limited support for systematically varying cut layers, data heterogeneity, and model choice within a single framework. As a result, reusable experimental pipelines and fair side-by-side comparisons remain difficult.

E. Positioning and scope

The proposed framework is designed as a research tool for systematic evaluation across split learning and split federated learning families under a shared infrastructure. It implements sequential and parallel SL baselines, canonical SFL-V1 and SFL-V2, and heterogeneity-aware extensions that support client-specific cut layers with layer-aware aggregation. The framework provides standardized configuration for model, architecture, client count, cut layers, and non-IID partitioning, and integrates logging and artifact generation to support reproducibility and comparative evaluation. Accordingly, SplitBench is positioned not only as an implementation artifact but also as reusable experimental infrastructure for SL/SFL research, enabling more consistent benchmarking and systematic comparison across diverse distributed learning settings.

Artifact	Contribution	Reference
SplitFed	Direct SL vs SFL-V1/V2 baselines in one codebase	[4], [19]
SLPerf	Benchmarking multiple SL paradigms systematically	[20]
SplitBud	Unified SL framework built on Flower	[21], [22]
PyVertical	Vertical SplitNN with PSI-based entity linkage	[23]
Flower	General FL substrate often extended for research frameworks	[24]
FedML	FL research library and benchmark for comparative studies	[25]

TABLE I: Representative implementations and frameworks relevant to SL/SFL experimentation

III. FRAMEWORK DESIGN AND IMPLEMENTATION

This section describes the design and implementation of the SL and SFL models supported by SplitBench. It presents the overall system structure, including the supported architectures, model abstraction layer, and data generation components, followed by the configuration, logging, and deployment mechanisms that support reproducible experimentation. The full code repository is available online [1].

A. System Architecture

SplitBench is organized as a set of modular components that collectively support configurable, reproducible, extensible, and reusable distributed training experiments for SL and SFL. The framework is structured around four main modules: an architecture module, a model registry, a data generation module, and a logging and reporting module. The architecture module encapsulates data transformers together with the definitions of client-side and server-side model components in an “architecture bundle,” while the model registry enables users to select and instantiate a deep learning architecture independently of the learning setup. Figure 1 illustrates the high-level system modules and their interactions.

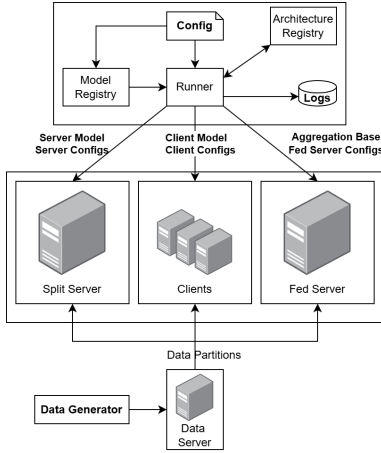


Fig. 1: Overview of the system architecture

B. Distributed Learning Architectures

SplitBench implements three categories of distributed learning architectures: base split learning, split federated learning, and custom cut split federated learning. This organization reflects increasing coordination, scalability, and

support for client heterogeneity while enabling controlled comparison within a common framework.

1) *Base Split Learning Architectures*: Base split learning architectures provide foundational execution models without federated aggregation. The framework supports two variants.

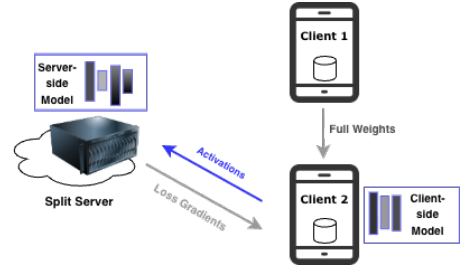


Fig. 2: Two-client split learning architecture

Round-Robin Split Learning implements a strictly sequential training process in which a single split server collaborates with multiple clients, activating one client at a time. Each client holds local data and the client-side portion of the model up to a fixed cut layer, while the server hosts the remaining layers. After a client completes training, its updated weights are passed to the next client, forming a chain of weight transfer. This architecture avoids parallelism and aggregation, providing a clear baseline for studying split execution and client-server interaction.

Thread-Based Split Learning enables parallel execution by assigning one server thread to each client. Each client connects to a dedicated server thread with its own server-side model instance and training loop. Clients train concurrently and independently, with no weight sharing or synchronization, providing a parallel SL baseline without collaborative aggregation.

2) *Split Federated Learning Architectures*: Split federated learning architectures combine split execution with federated aggregation for collaborative multi-client training.

Split Federated Learning V1 (SFL-V1) uses a dual-server design with a threaded split server and a separate federated server. Clients train in parallel through their assigned split server threads. At the end of each round, server-side submodels are aggregated on the split server and client-side submodels on the federated server, after which the aggregated models are redistributed for the next round.

Split Federated Learning V2 (SFL-V2) preserves the dual-server structure of SFL-V1 but replaces parallel server threads with a single shared server-side model. Clients are processed sequentially within each epoch, and client-side

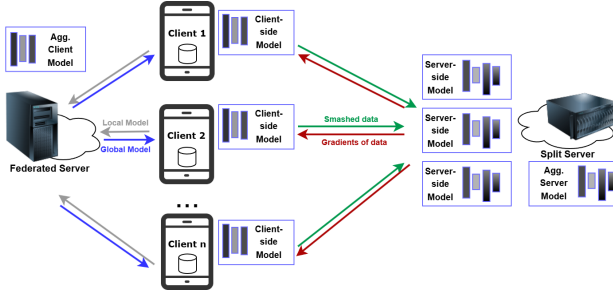


Fig. 3: Split federated learning architecture

models are aggregated on the federated server after each round. This reduces server-side memory and orchestration overhead at the cost of parallelism.

3) *Custom Cut Split Federated Learning Architectures*: Custom cut SFL architectures extend SFL-V1 and SFL-V2 to support client-specific cut-layer assignments.

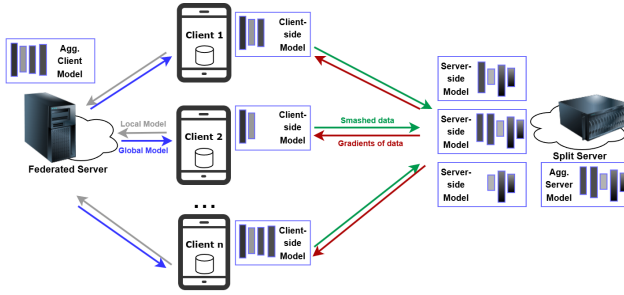


Fig. 4: Custom cut split federated learning architecture

Custom Cut SFL-V1 extends SFL-V1 by assigning each client its own cut layer. The split server spawns worker threads with client-specific server-side submodels, enabling parallel training under heterogeneous model partitions. Aggregation uses a layer-aware strategy in which only layers present in a client model participate, weighted by dataset size.

Custom Cut SFL-V2 generalizes sequential SFL-V2 to heterogeneous settings. A shared server-side model is dynamically reconfigured to match each client’s cut layer, and aggregation on the federated server follows the same layer-aware strategy as in Custom Cut SFL-V1. This trades parallelism for reduced server-side resource usage.

C. Model Abstraction and Registration

To decouple distributed learning algorithms from neural network design, SplitBench uses an “architecture bundle,” in which each neural network is represented as a sequence of layers that can be partitioned at arbitrary cut points. This allows the same SL or SFL workflow to operate across different model architectures without changing the training pipeline.

Each architecture consists of five components:

- a full reference model,
- a client-side submodel,
- a server-side submodel,

- a transformer for training data, and
- a transformer for evaluation data.

Client-side and server-side submodels are constructed automatically at runtime from the specified cut layer. Models are registered through a lightweight registry that supports automatic discovery and instantiation. Researchers can therefore add a new architecture in a single Python file while preserving compatibility with the training, logging, and evaluation pipeline.

D. Data Generation and Heterogeneity Control

The framework integrates a data generation module for controlled construction of IID and non-IID client datasets. Supported forms of heterogeneity include label skew, feature skew, and quantity skew.

SplitBench provides eight built-in partitioning strategies, including IID partitioning, shard-based non-IID partitioning, Dirichlet-based label and feature skew, percentage-based label skew, Gaussian feature skew, and two forms of Dirichlet-based quantity skew, following FedArtML [18]. It currently supports seven datasets: CIFAR-10, MNIST, DermaMNIST, DermaMNIST-corrected, and three OrganMNIST variants. Datasets and partitioning strategies are extensible through a modular data manager interface.

For Dirichlet-based and Gaussian-based skew settings, heterogeneity is controlled through a concentration parameter α , where smaller values produce stronger imbalance. This supports controlled evaluation from nearly IID to strongly skewed scenarios.

Generated data partitions can be visualized before training using built-in plotting utilities, helping relate observed training behavior to client-level data distributions.

E. Experiment Configuration, Logging, and Deployment

All experiments are driven by configuration files and command-line parameters specifying the learning architecture, number of clients, cut layers, dataset, data distribution parameters, and model architecture. This supports reproducibility and simplifies experiment management.

The framework includes structured logging of training metrics, communication events, and timing information for downstream analysis and comparison. It also supports containerized execution using Docker for repeatable deployment across environments.

Containerized deployment is supported for SFL variants and their custom cut counterparts through predefined Docker Compose configurations that simplify multi-container experiments and produce logs and model artifacts for downstream analysis.

IV. FRAMEWORK DEMONSTRATION BY EXPERIMENTS

This section demonstrates the capabilities of SplitBench through a representative set of experiments conducted with five clients across multiple datasets, models, and cut-layer settings. The goal is not to provide an exhaustive empirical comparison of all supported algorithms, but rather to illustrate how the framework enables controlled experimentation

under varying data heterogeneity, model selection, and client partition configurations. The examples also highlight the framework’s built-in support for visualization, logging, and reproducible analysis.

A. Data Heterogeneity and Partitioning

Building on the data generation and heterogeneity control mechanisms described in Section III-D, this subsection illustrates how SplitBench supports controlled experimental study of statistical heterogeneity through automatically generated partition visualizations.

To demonstrate this capability, we consider Dirichlet-based label skew on CIFAR-10 across five clients under two representative settings: $\alpha = 100$, which approximates IID partitioning, and $\alpha = 0.25$, which induces strong non-IID label imbalance.

Figure 5 shows the resulting per-client class distributions. Under $\alpha = 100$, each client receives a roughly balanced share of all ten classes. Under $\alpha = 0.25$, the class distribution becomes substantially more skewed, with some clients receiving a much larger share of particular classes while others are underrepresented. These plots are generated automatically by the framework’s built-in visualization utilities, allowing researchers to inspect partition characteristics before launching a full training run and to relate subsequent learning behavior to the underlying data distribution.

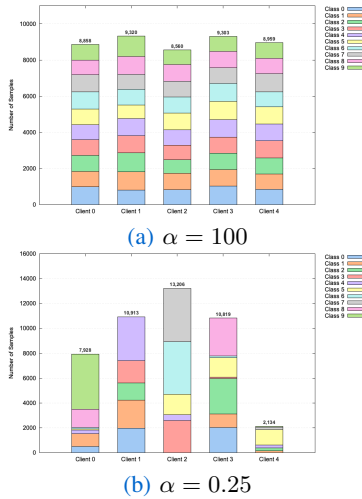


Fig. 5: Label-skew partitioning of CIFAR-10 across 5 clients

B. Model and Architecture Support

Building on the model abstraction and registration mechanism described in Section III-C, SplitBench enables controlled experimental comparison across different neural architectures without modifying the underlying distributed training workflow. The framework currently includes built-in architectures spanning multiple datasets, including ResNet18 and EfficientNet-B0 for DermaMNIST, ResNet18 and SimpleCNN for MNIST, and ResNet18 and EfficientNet for CIFAR-10.

One common use of the framework is to compare how different models behave under the same data conditions.

Figure 6 presents an example using CIFAR-10, comparing ResNet and EfficientNet under two heterogeneity levels, $\alpha = 100$ and $\alpha = 0.25$. By keeping the learning architecture and cut-layer configuration fixed, the framework makes it possible to attribute differences in the resulting learning curves to model choice and data heterogeneity rather than to changes in the experimental pipeline.

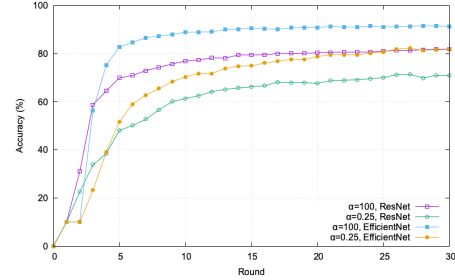


Fig. 6: ResNet and EfficientNet test accuracy on CIFAR-10

In addition to comparing model architectures, SplitBench supports experiments with heterogeneous client-side model partitions in the Custom Cut variants, where each client may operate with a different cut layer. This makes it possible to study how different partitioning choices affect collaborative training under shared experimental conditions.

Figure 7 shows test accuracy curves produced by the framework for Custom Cut SFL-V1 on CIFAR-10 over 30 rounds using both ResNet18 and EfficientNet. Three cut-layer configurations are compared: (4, 4, 4, 4, 4), (2, 4, 4, 4, 6), and (3, 3, 4, 5, 5). The first configuration represents a homogeneous setting in which all clients use the same partition point. The second represents a partially heterogeneous setting in which most clients share a common cut layer, while one client uses a shallower partition and another uses a deeper one. The third represents a more balanced heterogeneous setting in which no single client profile dominates. The curves are generated automatically from the recorded server logs, illustrating how the framework supports direct comparison across cut-layer assignments and model choices. While this paper is mainly focused on discussing the features of SplitBench, more analysis regarding the effect of data partitioning and also the effect of cut-layer choice can be found in our previous work [11], [12].

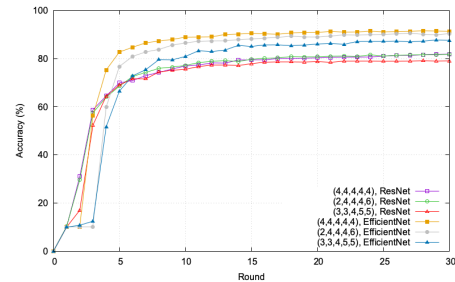


Fig. 7: Test accuracy under model and cut-layer variations

C. Logging and Reproducibility

The shown figures are generated directly from data recorded in the experiment logs, illustrating how SplitBench supports reproducible analysis without additional manual post-processing. Each experiment produces a self-contained run directory that captures the information needed to reproduce, inspect, or audit the results. This includes separate log files for every process, model checkpoints after each round, per-class accuracy, sample counts, and full confusion matrices for both validation and test sets. In addition, round-level summaries report timing breakdowns, communication statistics, and class-wise diagnostic information.

The timing summaries include quantities such as training time, server-side work time, and federated waiting time, while the communication summaries record the number of bytes exchanged between clients and servers. Class-wise diagnostics are available after every round, enabling researchers to monitor not only aggregate performance but also which classes remain underserved during training.

At the end of a run, the framework identifies the best-performing rounds with their corresponding per-class accuracy breakdowns. This makes it easier to locate suitable checkpoints and to examine where overfitting may begin to emerge without re-evaluating saved models. The structured log format is compatible with standard downstream analysis tools and supports side-by-side comparison across experiments that vary in data heterogeneity, model architecture, cut-layer assignment, or learning setup.

V. CONCLUSIONS AND FUTURE WORK

This paper presented SplitBench, an open-source framework for unified and reusable benchmarking of split learning and split federated learning within a single configurable codebase. The framework supports custom cut layer selection for clients, while decoupling architecture, model, and dataset selection from training logic to enable systematic and reproducible comparison across experimental conditions. It also supports controlled non-IID data generation, built-in visualization, extensible model integration through the architecture bundle abstraction, and containerized deployment for portable experimentation. Future work includes extending the framework with additional split learning topologies, and resource-aware heuristics for cut-layer selection.

REFERENCES

- [1] Y. Amannejad and collaborators, “SplitBench Repository.” <https://github.com/Yasaman-A/split-learning>, 2026. Accessed: June 2026.
- [2] P. Vepakomma, O. Gupta, T. Swedish, and R. Raskar, “Split learning for health: Distributed deep learning without sharing raw patient data,” *arXiv preprint arXiv:1812.00564*, 2018.
- [3] G. Yansong, M. Kim, S. Abuadbba, et al., “End-to-end evaluation of federated learning and split learning for internet of things,” in *International Symposium on Reliable Distributed Systems (SRDS)*, vol. 4, 2020.
- [4] C. Thapa, P. C. M. Arachchige, S. Camtepe, and L. Sun, “Splitfed: When federated learning meets split learning,” in *AAAI Conference on Artificial Intelligence*, vol. 36, pp. 8485–8493, 2022.
- [5] Z. Lin, G. Zhu, Y. Deng, X. Chen, Y. Gao, K. Huang, and Y. Fang, “Efficient parallel split learning over resource-constrained wireless edge networks,” *IEEE Transactions on Mobile Computing*, vol. 23, no. 10, pp. 9224–9239, 2024.
- [6] M. Kohankhaki, A. Ayad, M. Barhoush, and A. Schmeink, “Parallel split learning with global sampling,” in *2025 3rd International Conference on Foundation and Large Language Models (FLLM)*, p. 281–288, IEEE, Nov. 2025.
- [7] S. Lyu, Z. Lin, G. Qu, X. Chen, X. Huang, and P. Li, “Optimal resource allocation for u-shaped parallel split learning in edge networks,” 2023.
- [8] H. Tang, Z. Zhao, D. Liu, Y. Cao, S. Zhang, and S. You, “Edge-assisted u-shaped split federated learning with privacy-preserving for internet of things,” 2023.
- [9] C. Thapa, M. A. P. Chamikara, S. Camtepe, and L. Sun, “Splitfed: When federated learning meets split learning,” 2022.
- [10] J. Zheng, Y. Chen, and Q. Lai, “PPSFL: Privacy-Preserving Split Federated Learning for heterogeneous data in edge-based Internet of Things,” *Future Generation Computer Systems*, vol. 156, pp. 231–241, 2024.
- [11] S. Trepac and Y. Amannejad, “Performance observations from split federated learning on heterogeneous devices and networks,” in *IEEE NOMS*, pp. 1–7, 2025.
- [12] S. Trepac and Y. Amannejad, “Split federated learning under joint system and data heterogeneity,” *IEEE Access*, 2026.
- [13] Y. Zhao, M. Li, L. Lai, N. Suda, D. Cavin, and V. Chandra, “Federated learning with non iid data,” *arXiv preprint arXiv:1806.00582*, 2018.
- [14] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, “Federated optimization in heterogeneous networks,” in *Proceedings of Machine Learning and Systems (MLSys)*, 2020.
- [15] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, “Scaffold: Stochastic controlled averaging for federated learning,” in *International conference on machine learning*, pp. 5132–5143, PMLR, 2020.
- [16] T.-M. H. Hsu, H. Qi, and M. Brown, “Measuring the effects of non-iid data on federated learning,” in *NeurIPS Workshop on Federated Learning*, 2019.
- [17] M. Yurochkin, M. Agarwal, S. Ghosh, K. Greenewald, T. N. Hoang, and Y. Khazaeni, “Bayesian nonparametric federated learning of neural networks,” 2019.
- [18] D. M. J. Gutierrez, A. Anagnostopoulos, I. Chatzigiannakis, and A. Vitaletti, “Fedartml: A tool to facilitate the generation of non-iid datasets for federated learning research,” *IEEE Access*, vol. 12, pp. 81004–81016, 2024.
- [19] C. Thapa and collaborators, “Splitfed: When federated learning meets split learning (reference implementation).” GitHub repository, 2020. <https://github.com/chandra2thapa/SplitFed-When-Federated-Learning-Meets-Split-Learning>.
- [20] T. Zhou, Z. Hu, B. Wu, and C. Chen, “SLPerf: a unified framework for benchmarking split learning,” *arXiv preprint arXiv:2304.01502*, 2023.
- [21] B. Radović, M. Canini, S. Horváth, V. Pejović, and P. Vepakomma, “Towards a unified framework for split learning,” in *EuroMLSys*, 2025. Note: Associated with the SplitBud framework.
- [22] S. Lab and contributors, “SplitBud: Split learning framework built on top of flower.” GitHub repository, 2025. <https://github.com/sands-lab/slower>.
- [23] D. Romanini, A. J. Hall, P. Papadopoulos, T. Titcombe, A. Ismail, T. Cebere, R. Sandmann, R. Roehm, and M. A. Hoeh, “Pyvertical: A vertical federated learning framework for multi-headed splitnn,” *arXiv preprint arXiv:2104.00489*, 2021.
- [24] D. J. Beutel, T. Topal, A. Mathur, X. Qiu, J. Fernandez-Marques, Y. Gao, L. Sani, K. H. Li, T. Parcollet, P. P. B. de Gusmão, and N. D. Lane, “Flower: A friendly federated learning research framework,” *arXiv preprint arXiv:2007.14390*, 2020.
- [25] C. He, S. Li, J. So, X. Zeng, M. Zhang, H. Wang, X. Wang, P. Vepakomma, A. Singh, H. Qiu, X. Zhu, J. Wang, L. Shen, P. Zhao, Y. Kang, Y. Liu, R. Raskar, Q. Yang, M. Annamalai, and S. Avestimehr, “Fedml: A research library and benchmark for federated machine learning,” 2020.
- [26] Z. Hu, T. Zhou, B. Wu, C. Chen, and Y. Wang, “A review and experimental evaluation on split learning,” *Future Internet*, vol. 17, no. 2, p. 87, 2025.
- [27] Y. Liu, T. Fan, T. Chen, Q. Xu, and Q. Yang, “Fate: An industrial grade platform for collaborative learning with federated learning,” *Journal of Machine Learning Research*, vol. 22, no. 226, pp. 1–6, 2021.